

Column-Oriented Databases

1. Overview

Column-oriented databases store data by **columns instead of rows**.

Traditional row-based databases store:

None

```
Row1 = [id, name, age, city]
```

```
Row2 = [id, name, age, city]
```

Column-oriented systems store:

None

```
id    = [1,2,3]
```

```
name  = [A,B,C]
```

```
age   = [21,25,30]
```

```
city  = [NY,LA,SF]
```

This design is highly efficient for:

- Filtering specific columns
- Aggregations
- Large-scale analytics
- Sparse distributed data
- High write throughput (some systems)

2. Important Clarification

There are **two related meanings** often confused:

Meaning	Examples
Analytical columnar storage	ClickHouse, Apache Parquet
Wide-column NoSQL stores	Apache Cassandra, Apache HBase

Your example (Cassandra, HBase) belongs mainly to **wide-column distributed NoSQL systems**.

3. Row Store vs Column Store

Row-Based Storage

Good when reading complete records.

Example:

None

```
Get user id=101 full profile
```

Column-Based Storage

Good when reading few columns from huge datasets.

Example:

None

```
Find average salary of 10 million users
```

```
Need only salary column
```

4. Why Column Storage Is Fast

If query needs only 1–2 columns:

SQL

```
SELECT AVG(price) FROM products;
```

No need to read name, description, image, etc.

Reads less data → faster scans.

5. Compression Benefits

Same-type values stored together.

Example:

None

country column:

IN IN IN IN US US US

Compresses very efficiently.

This improves:

- Storage savings
- Faster reads
- Better cache efficiency

6. Use Cases

Best for:

- Analytics
- Reporting
- OLAP workloads
- Metrics systems
- Data warehousing
- Event analysis
- Large filtering queries

7. Wide-Column Databases (Cassandra / HBase)

These are distributed NoSQL systems inspired by column families.

Instead of strict SQL rows/tables, they organize data into:

- Row keys
- Column families
- Dynamic columns

Examples:

- Apache Cassandra
- Apache HBase

Best for:

- Massive writes
- Time-series data
- Large distributed datasets
- High availability

8. Example Wide-Column Record

None

Row Key: user_101

profile:name = Samarth

profile:city = Surat

activity:last_login = today

activity:orders = 14

Flexible schema.

9. Column Family Concept

Related columns grouped together.

None

profile family:

name, email, city

activity family:

login_time, purchases

Efficient for distributed reads.

10. Analytical Columnar Examples

Used in BI / warehouses:

- ClickHouse
- Snowflake
- Google BigQuery

Great for:

None
COUNT
SUM
AVG
GROUP BY
large scans

11. Column Store vs Row Store

Feature	Row Store	Column Store
Full record lookup	Excellent	Good
Analytics scans	Slower	Excellent

Compression	Moderate	High
OLTP transactions	Excellent	Usually weaker
Insert many rows	Good	Varies

12. Example Queries

Better for Column Store

SQL

```
SELECT SUM(revenue)
FROM sales
WHERE region='West';
```

Better for Row Store

SQL

```
SELECT *
FROM users
WHERE user_id=101;
```

13. Why Cassandra Is Popular

- Distributed by design
- Multi-node replication
- High availability
- Fast writes
- Handles large scale

Used in:

- Messaging systems
- Telemetry
- Feed systems
- IoT data

14. Why HBase Is Popular

Built on Apache Hadoop HDFS ecosystem.

Good for:

- Very large sparse tables
- Batch + random access workloads

15. Trade-offs

Complex schema design around access patterns

Joins weak or absent

Eventual consistency in some systems

Harder ad hoc relational queries

16. Interview Answer Example

Q: Why choose Cassandra?

I'd choose Cassandra because we need horizontally scalable writes, high availability, and predictable performance for large time-series/event workloads.

17. Short Revision Notes

None

Column-oriented = stores data by columns

Benefits:

- Fast filtering
- Fast aggregation
- High compression

Examples:

- Cassandra
- HBase
- ClickHouse

18. One-Line Summary

Column-oriented databases organize data by columns rather than rows, enabling faster analytics, filtering, and scalable large-data workloads, while wide-column systems like Cassandra and HBase add distributed NoSQL scalability.